

Домашнее задание

Загрузка данных в ClickHouse

ETL-пайплайн: Open-Meteo API -> Apache Airflow -> ClickHouse

Дата выполнения: 01.07.2026

Параметр	Значение
Инструмент переливки данных	Apache Airflow 2.9.3 в Docker
Целевая СУБД	ClickHouse Server 25.8 в Docker
Источник данных	Open-Meteo Historical Weather API
Тип данных	Ежедневные погодные показатели по городам
Таблица ClickHouse	nasa.weather_daily
Режим запуска	Регулярный запуск по расписанию @daily и ручной запуск через Airflow UI

1. Цель работы

Целью домашнего задания является настройка ETL-процесса для загрузки данных из внешнего источника в ClickHouse. В работе был развернут Apache Airflow как инструмент оркестрации, ClickHouse как целевая аналитическая база данных, а также построен DAG для регулярной загрузки данных из внешнего API.

2. Архитектура решения

Для выполнения задания использовалась следующая схема:

```
Open-Meteo Historical Weather API
    ↓
Apache Airflow DAG open_meteo_to_clickhouse
    ↓
PythonOperator: requests.get() + clickhouse_driver
    ↓
ClickHouse table nasa.weather_daily
```

Стенд развернут с помощью Docker Compose на виртуальной машине. В составе стенда запущены контейнеры ClickHouse, Airflow Webserver и Airflow Scheduler.

3. Соответствие требованиям задания

Требование	Реализация
Развернуть инструмент для переливки данных	Развернут Apache Airflow в Docker: webserver и scheduler.
Развернуть ClickHouse	Развернут контейнер clickhouse/clickhouse-server:25.8.
Выбрать источник данных	Выбран Open-Meteo Historical Weather API.
Настроить подключение к ClickHouse	DAG подключается к ClickHouse через библиотеку clickhouse_driver.
Построить пайплайн	Создан DAG open_meteo_to_clickhouse с двумя задачами: создание таблицы и загрузка данных.
Регулярное или одноразовое выполнение	DAG имеет schedule="@daily" и также запускается вручную через Airflow UI.

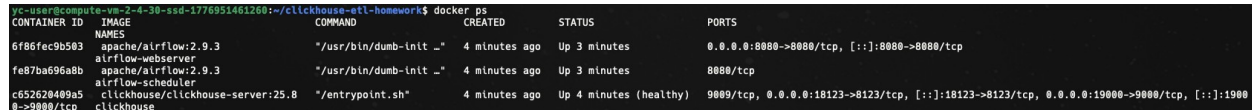
4. Развертывание компонентов

Контейнеры были запущены командой:

```
docker compose up -d
```

Проверка работающих контейнеров выполнялась командой:

```
docker ps
```



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
6f86fec9b503	apache/airflow:2.9.3	"/usr/bin/dumb-init ..."	4 minutes ago	Up 3 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
fe87ba96a8b	apache/airflow:2.9.3	"/usr/bin/dumb-init ..."	4 minutes ago	Up 3 minutes	8080/tcp
c652620489a5	airflow-scheduler	"/entrypoint.sh"	4 minutes ago	Up 4 minutes (healthy)	9009/tcp, 0.0.0.0:18123->8123/tcp, [::]:18123->8123/tcp, 0.0.0.0:19000->9000/tcp, [::]:19000->9000/tcp
8->9000/tcp	clickhouse				

Рисунок 1 - Контейнеры Airflow и ClickHouse запущены в Docker

5. Структура таблицы ClickHouse

В ClickHouse используется база `паса` и таблица `weather_daily`. Таблица хранит дневные погодные показатели по городам.

```
CREATE TABLE IF NOT EXISTS weather_daily
(
    date Date,
    city LowCardinality(String),
    latitude Float64,
    longitude Float64,
    temperature_2m_max Float64,
    temperature_2m_min Float64,
    precipitation_sum Float64,
    wind_speed_10m_max Float64,
    loaded_at DateTime DEFAULT now()
)
ENGINE = MergeTree
ORDER BY (city, date);
```

В качестве ключа сортировки выбран набор полей (`city, date`), так как аналитические запросы обычно выполняются по городу и периоду дат.

6. Airflow DAG

DAG называется `open_meteo_to_clickhouse`. Он содержит две задачи:

- `create_clickhouse_table` - создает таблицу в ClickHouse, если она отсутствует;
- `load_weather_to_clickhouse` - получает данные из API и вставляет их в ClickHouse.

Фрагмент настройки DAG:

```
with DAG(
    dag_id="open_meteo_to_clickhouse",
    description="Load daily weather data from Open-Meteo API to ClickHouse",
    start_date=datetime(2026, 6, 25),
    schedule="@daily",
    catchup=True,
    tags=["etl", "clickhouse", "weather", "api"],
) as dag:
    create_clickhouse_table >> load_weather_to_clickhouse
```

7. Выполнение пайплайна

После загрузки DAG в Airflow он был включен и запущен вручную. В интерфейсе Airflow видно, что DAG активен, а последние задачи завершились успешно.

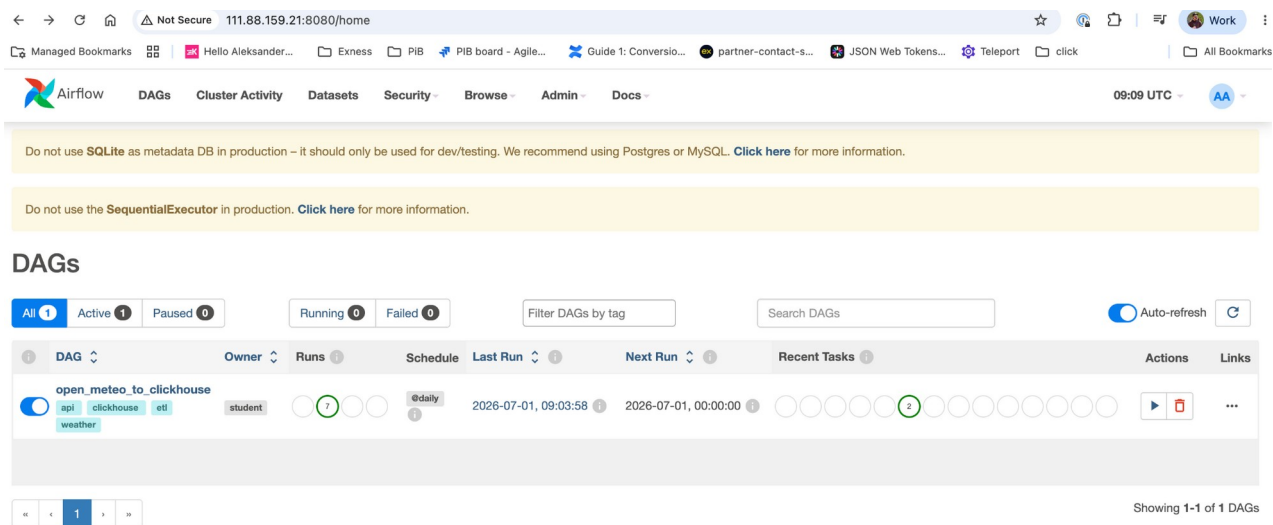


Рисунок 2 - DAG `open_meteo_to_clickhouse` активен и успешно выполнен

8. Проверка данных в ClickHouse

Для проверки загруженных данных был выполнен SQL-запрос:

```
SELECT *
FROM weather_daily
ORDER BY date DESC, city
LIMIT 20;
```

```
yc-user@compute-vm-2-4-30-ssd-1776951461260:~/clickhouse-etl-homework$ docker exec -it clickhouse clickhouse-client \
--user airflow \
--password airflow_password \
--database nasa
ClickHouse client version 25.8.25.37 (official build).
Connecting to database nasa at localhost:9000 as user airflow.
Connected to ClickHouse server version 25.8.25.

Warnings:
* Delay accounting is not enabled, OSIOWaitMicroseconds will not be gathered. You can enable it using 'sudo sh -c 'echo 1 > /proc/sys/kernel/task_delayacct'' or by using sysctl.

c552620489a5 ) SELECT *
FROM weather_daily
ORDER BY date DESC, city
LIMIT 20;

SELECT *
FROM weather_daily
ORDER BY
  date DESC,
  city ASC
LIMIT 20

Query id: 7f8246ac-5252-4e31-bd2b-5370031941b7

+----+----+----+----+----+----+----+----+----+----+
| date                | city                | latitude | longitude | temperature_2m_max | temperature_2m_min | precipitation_sum | wind_speed_10m_max | loaded_at                |
+----+----+----+----+----+----+----+----+----+----+
| 2026-07-01          | Moscow              | 55.7558  | 37.6173   | 28.1               | 18.9               | 0.1               | 11.4                | 2026-07-01 09:08:27     |
| 2026-07-01          | Novosibirsk         | 55.0084  | 82.9357   | 29.1               | 17.4               | 0                 | 10.3                | 2026-07-01 09:08:27     |
| 2026-07-01          | Saint Petersburg    | 59.9343  | 30.3351   | 25.9               | 17                 | 0.1               | 12.8                | 2026-07-01 09:08:27     |
| 2026-06-30          | Moscow              | 55.7558  | 37.6173   | 26.7               | 14.7               | 0                 | 12                 | 2026-07-01 09:07:01     |
| 2026-06-30          | Novosibirsk         | 55.0084  | 82.9357   | 28                 | 18.8               | 7.3               | 15.4                | 2026-07-01 09:07:01     |
| 2026-06-30          | Saint Petersburg    | 59.9343  | 30.3351   | 25.1               | 17                 | 0.4               | 19.4                | 2026-07-01 09:07:01     |
| 2026-06-29          | Moscow              | 55.7558  | 37.6173   | 22.4               | 15.7               | 0.1               | 12                 | 2026-07-01 09:06:54     |
| 2026-06-29          | Novosibirsk         | 55.0084  | 82.9357   | 30.2               | 18.7               | 0.8               | 14.5                | 2026-07-01 09:06:54     |
| 2026-06-29          | Saint Petersburg    | 59.9343  | 30.3351   | 23                 | 18.4               | 9.8               | 13                 | 2026-07-01 09:06:54     |
| 2026-06-28          | Moscow              | 55.7558  | 37.6173   | 22.8               | 13.5               | 0.5               | 17.2                | 2026-07-01 09:06:46     |
| 2026-06-28          | Novosibirsk         | 55.0084  | 82.9357   | 27.1               | 18                 | 2.3               | 12.1                | 2026-07-01 09:06:46     |
| 2026-06-28          | Saint Petersburg    | 59.9343  | 30.3351   | 26.7               | 14.2               | 1.6               | 11.2                | 2026-07-01 09:06:46     |
| 2026-06-27          | Moscow              | 55.7558  | 37.6173   | 20.6               | 12.5               | 1.1               | 13                 | 2026-07-01 09:08:41     |
| 2026-06-27          | Novosibirsk         | 55.0084  | 82.9357   | 27.6               | 18.7               | 0.5               | 15.1                | 2026-07-01 09:08:41     |
| 2026-06-27          | Saint Petersburg    | 59.9343  | 30.3351   | 20.1               | 15.1               | 3.1               | 14.3                | 2026-07-01 09:08:41     |
| 2026-06-26          | Moscow              | 55.7558  | 37.6173   | 20.3               | 11.7               | 1.9               | 15.5                | 2026-07-01 09:08:36     |
| 2026-06-26          | Novosibirsk         | 55.0084  | 82.9357   | 28.9               | 19.7               | 2.2               | 14.1                | 2026-07-01 09:08:36     |
| 2026-06-26          | Saint Petersburg    | 59.9343  | 30.3351   | 19.8               | 13.3               | 2.1               | 20.3                | 2026-07-01 09:08:36     |
| 2026-06-25          | Moscow              | 55.7558  | 37.6173   | 19.3               | 13.8               | 2.2               | 11.4                | 2026-07-01 09:08:21     |
| 2026-06-25          | Novosibirsk         | 55.0084  | 82.9357   | 30.8               | 19.2               | 0                 | 10.5                | 2026-07-01 09:08:21     |
+----+----+----+----+----+----+----+----+----+----+

20 rows in set. Elapsed: 0.039 sec.
```

Рисунок 3 - Данные о погоде загружены в таблицу `weather_daily`

9. Аналитический запрос

Для демонстрации работы с загруженными данными была выполнена агрегация по городам:

```
SELECT
    city,
    count() AS days_loaded,
    round(avg(temperature_2m_max), 2) AS avg_max_temp,
    round(avg(temperature_2m_min), 2) AS avg_min_temp,
    round(sum(precipitation_sum), 2) AS total_precipitation
FROM weather_daily
GROUP BY city
ORDER BY city ASC;
```



```
c652620489a5 :) SELECT
    city,
    count() AS days_loaded,
    round(avg(temperature_2m_max), 2) AS avg_max_temp,
    round(avg(temperature_2m_min), 2) AS avg_min_temp,
    round(sum(precipitation_sum), 2) AS total_precipitation
FROM weather_daily
GROUP BY city
ORDER BY city;

SELECT
    city,
    count() AS days_loaded,
    round(avg(temperature_2m_max), 2) AS avg_max_temp,
    round(avg(temperature_2m_min), 2) AS avg_min_temp,
    round(sum(precipitation_sum), 2) AS total_precipitation
FROM weather_daily
GROUP BY city
ORDER BY city ASC

Query id: 618ae81a-2515-45b1-850a-00df3050a4de

+----+-----+-----+-----+-----+
| city | days_loaded | avg_max_temp | avg_min_temp | total_precipitation |
+----+-----+-----+-----+-----+
| 1. Moscow | 7 | 22.89 | 14.34 | 5.9 |
| 2. Novosibirsk | 7 | 28.81 | 18.64 | 13.1 |
| 3. Saint Petersburg | 7 | 23.1 | 15.27 | 17.8 |
+----+-----+-----+-----+-----+

3 rows in set. Elapsed: 0.049 sec.
```

Рисунок 4 - Агрегация погодных данных по городам

По результатам запроса в таблице содержатся данные за 7 дней по каждому из трех городов: Москва, Новосибирск и Санкт-Петербург.

10. Результаты выполнения

Город	Дней загружено	Средняя max t, C	Средняя min t, C	Сумма осадков
Moscow	7	22.89	14.34	5.9
Novosibirsk	7	28.81	18.64	13.1
Saint Petersburg	7	23.10	15.27	17.8

11. Вывод

В ходе работы был настроен ETL-процесс загрузки данных из внешнего API в ClickHouse. В качестве инструмента оркестрации использован Apache Airflow, развернутый в Docker. В качестве целевой базы данных использован ClickHouse. Источником данных выступил Open-Meteo Historical Weather API.

Пайплайн поддерживает регулярное выполнение по расписанию @daily и ручной запуск из интерфейса Airflow. После запуска DAG данные успешно появились в таблице ClickHouse weather_daily, что подтверждается SQL-запросами и результатами агрегации.

Приложение А. Полный код DAG

```
from __future__ import annotations

from datetime import datetime, timedelta, date

import requests
from airflow import DAG
from airflow.operators.python import PythonOperator
from clickhouse_driver import Client

CLICKHOUSE_HOST = "clickhouse"
CLICKHOUSE_PORT = 9000
CLICKHOUSE_USER = "airflow"
CLICKHOUSE_PASSWORD = "airflow_password"
CLICKHOUSE_DATABASE = "nasa"

def get_clickhouse_client() -> Client:
    return Client(
        host=CLICKHOUSE_HOST,
        port=CLICKHOUSE_PORT,
        user=CLICKHOUSE_USER,
        password=CLICKHOUSE_PASSWORD,
        database=CLICKHOUSE_DATABASE,
    )

def create_table() -> None:
    client = get_clickhouse_client()
    client.execute("""
        CREATE TABLE IF NOT EXISTS weather_daily
        (
            date Date,
            city LowCardinality(String),
            latitude Float64,
            longitude Float64,
            temperature_2m_max Float64,
            temperature_2m_min Float64,
            precipitation_sum Float64,
            wind_speed_10m_max Float64,
            loaded_at DateTime DEFAULT now()
        )
        ENGINE = MergeTree
        ORDER BY (city, date)
    """)

def load_weather(**context) -> None:
    logical_date = context["ds"]
    cities = [
        {"city": "Moscow", "latitude": 55.7558, "longitude": 37.6173},
        {"city": "Saint Petersburg", "latitude": 59.9343, "longitude": 30.3351},
        {"city": "Novosibirsk", "latitude": 55.0084, "longitude": 82.9357},
    ]
    rows = []

    for city in cities:
        response = requests.get(
            "https://archive-api.open-meteo.com/v1/archive",
            params={
                "latitude": city["latitude"],
                "longitude": city["longitude"],
                "start_date": logical_date,
                "end_date": logical_date,
                "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum,wind_speed_10m_max",
            },
        )
```

```

        "timezone": "Europe/Moscow",
    },
    timeout=30,
)
response.raise_for_status()
daily = response.json()["daily"]
rows.append((
    date.fromisoformat(daily["time"][0]),
    city["city"],
    city["latitude"],
    city["longitude"],
    float(daily["temperature_2m_max"][0]),
    float(daily["temperature_2m_min"][0]),
    float(daily["precipitation_sum"][0]),
    float(daily["wind_speed_10m_max"][0]),
))

client = get_clickhouse_client()
client.execute("""
    INSERT INTO weather_daily
    (
        date,
        city,
        latitude,
        longitude,
        temperature_2m_max,
        temperature_2m_min,
        precipitation_sum,
        wind_speed_10m_max
    )
    VALUES
""", rows)

default_args = {
    "owner": "student",
    "retries": 2,
    "retry_delay": timedelta(minutes=2),
}

with DAG(
    dag_id="open_meteo_to_clickhouse",
    description="Load daily weather data from Open-Meteo API to ClickHouse",
    default_args=default_args,
    start_date=datetime(2026, 6, 25),
    schedule="@daily",
    catchup=True,
    tags=["etl", "clickhouse", "weather", "api"],
) as dag:
    create_clickhouse_table = PythonOperator(
        task_id="create_clickhouse_table",
        python_callable=create_table,
    )
    load_weather_to_clickhouse = PythonOperator(
        task_id="load_weather_to_clickhouse",
        python_callable=load_weather,
    )
    create_clickhouse_table >> load_weather_to_clickhouse

```